SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL1- Constraint-Based Problem Solving**
Weightage:	**40%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	N.AKHIL KUMAR	**Reg.No.:**	192425128
Department:	AIML	**Date:**	

ANNEXURE A

Constraint-Based Problem Solving

1.Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3.Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4.Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5.Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 µs
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I N.AKHIL KUMAR(192425128) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N.AKHIL KUMAR

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ANSWERS:

Q1. Real-Time Embedded I/O Communication Mechanism

1. Problem Understanding

The embedded system receives data from multiple sensors. Each sensor generates an interrupt every 1 ms, so the processor must respond to frequent I/O events while keeping CPU utilization below 15%.

The system must:

- Handle multiple high-frequency sensor interrupts.
- Use vectored interrupt handling.
- Minimize interrupt-service overhead.
- Select an appropriate I/O addressing mechanism.
- Maintain deterministic real-time response.
- Keep CPU utilization below 15%.

If there are N sensors, each generating one interrupt every 1 ms:

[
Interrupt\ Rate = N \times 1000\ interrupts/sec
]

For example, with 8 sensors:

[
8 \times 1000 = 8000\ interrupts/sec
]

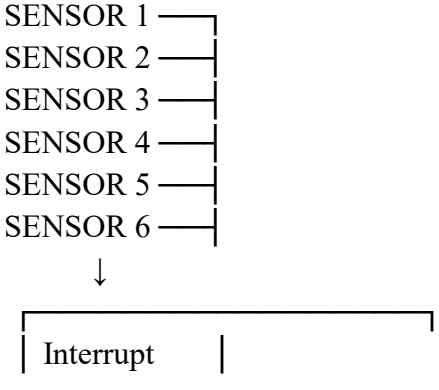
Therefore, servicing every interrupt with a long ISR would unnecessarily consume CPU time.

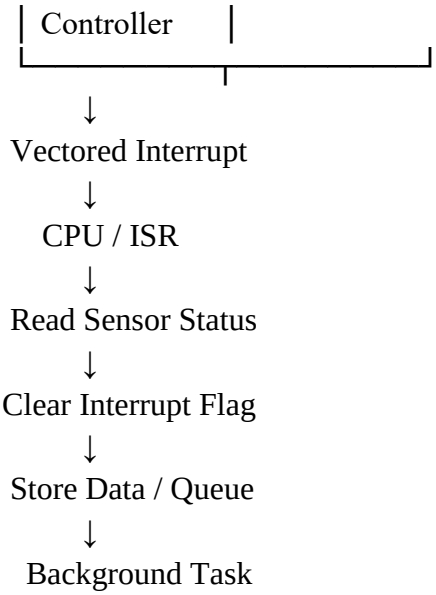
2. Constraint Analysis

Constraint	Design Requirement
CPU utilization < 15%	Keep ISR extremely short
Interrupt every 1 ms	High interrupt frequency
Multiple sensors	Need efficient prioritization
Vectored interrupts	Direct ISR dispatch
Low latency	Use memory-mapped I/O
Real-time operation	Predictable scheduling

The most important constraint is the CPU utilization limit. Therefore, the system should avoid performing complete sensor processing inside the ISR.

3. Proposed Architecture





4. Memory-Mapped vs I/O-Mapped I/O

Memory-Mapped I/O

Device registers are assigned addresses in the processor's normal memory address space.

CPU
↓
Memory Address
↓
Sensor Register

Advantages:

- Simple programming model.
- Normal load/store instructions can access registers.
- Convenient for modern RISC processors.
- Easy integration with DMA and memory protection mechanisms.
- Low software overhead.

I/O-Mapped I/O

Separate I/O address space is used.

CPU
↓
I/O Instruction
↓
Device Register

It can be useful on architectures specifically designed around separate I/O spaces, but it requires dedicated I/O instructions.

Selection

Memory-mapped I/O is recommended for this design because it provides a simple, low-overhead register-access mechanism and is well suited to modern embedded processors.

5. Vectored Interrupt Handling

Instead of using one common ISR:

Interrupt
↓
Common ISR

↓
Identify Sensor
↓
Call Handler
the system uses:
Sensor 1 → Vector 1 → ISR1
Sensor 2 → Vector 2 → ISR2
Sensor 3 → Vector 3 → ISR3
Sensor 4 → Vector 4 → ISR4
This reduces software dispatch overhead.

6. CPU Utilization Strategy

The ISR should perform only essential operations:
ISR:
1. Save minimum required context
2. Read sensor status
3. Read/queue sensor data
4. Clear interrupt
5. Signal processing task
6. Return
Expensive operations such as filtering, machine-learning inference, or sensor fusion should execute outside the ISR.
If:

- N = number of sensors
- F = interrupt frequency per sensor
- T_ISR = ISR execution time

then approximate interrupt CPU utilization is:
[
U = N \times F \times T_{ISR}
]
The design must ensure:
[
U < 0.15
]
Example
For 8 sensors:
[
F=1000\ interrupts/sec
]
Suppose:
[
T_{ISR}=10\ \mu s
]
Then:
[
U=8 \times 1000 \times 10\mu s
]

[
U=0.08=8%
]
Thus, the interrupt-processing portion remains below 15%, leaving CPU capacity for normal application processing.

7. Justification
The recommended design combines:
Memory-mapped I/O + vectored interrupts + short ISRs + deferred processing.
This minimizes latency and CPU utilization while allowing the system to handle frequent sensor interrupts.

Q2. High-Speed NVMe I/O Subsystem

1. Problem Understanding

The system must support storage throughput greater than 3 GB/s using NVMe over PCIe. The CPU must not handle bulk data movement.
Therefore, the design requires:

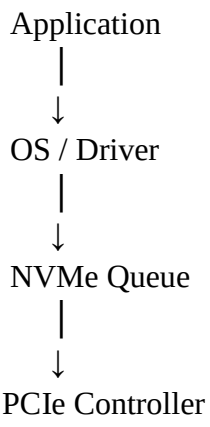
- NVMe storage.
- PCIe interface.
- DMA.
- Interrupt-based completion notification.
- High-throughput memory transfers.
- Minimal CPU intervention.

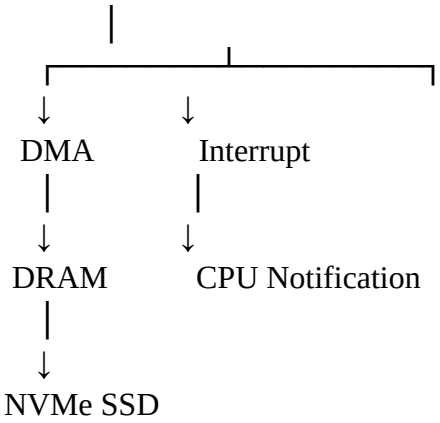
2. Constraint Analysis

Constraint	Required Solution
>3 GB/s	High-speed PCIe/NVMe
NVMe over PCIe	PCIe NVMe controller
CPU should not copy bulk data	DMA
Completion notification	MSI/MSI-X interrupts
High throughput	Queue-based architecture
Low CPU overhead	Batched completions

The key requirement is to remove the CPU from the bulk data path.

3. Proposed Architecture





The CPU prepares commands, but the actual storage-to-memory transfer is performed using DMA.

4. DMA Operation

Read Operation

NVMe SSD
↓
PCIe
↓
DMA Controller
↓
DRAM
↓
Application

The CPU does not copy every byte.

It only:

1. Creates/submits the I/O request.
2. Provides buffer information.
3. Starts the operation.
4. Receives completion notification.
5. Processes the result.

5. Interrupt Notification

Modern NVMe devices commonly use MSI/MSI-X mechanisms for completion notification.

Multiple queues can be associated with different CPUs or processing contexts:

CPU 0 ← Queue 0

CPU 1 ← Queue 1

CPU 2 ← Queue 2

CPU 3 ← Queue 3

This improves parallelism and reduces contention.

6. Throughput Analysis

The required throughput is:

[

T > 3\ GB/s

]

The PCIe link must therefore provide enough usable bandwidth after protocol overhead.

For example, a PCIe configuration with sufficient lane count and generation should be selected so that its practical payload bandwidth comfortably exceeds the required 3 GB/s.
The design should benchmark:

- Sequential read throughput.
- Sequential write throughput.
- Random I/O.
- Queue depth.
- DMA efficiency.
- CPU utilization.

7. Justification

DMA is essential because transferring every byte through CPU instructions would consume significant processor resources.
The proposed:
NVMe + PCIe + DMA + MSI/MSI-X + multiple queues
architecture allows high throughput while keeping CPU involvement low.

Q3. Multi-Device USB and Thunderbolt Communication Architecture

1. Problem Understanding

The system contains:

- 6 USB peripherals
- 2 high-speed Thunderbolt devices

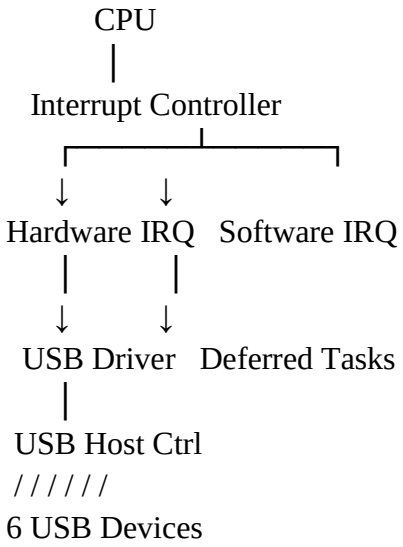
The architecture must prevent:

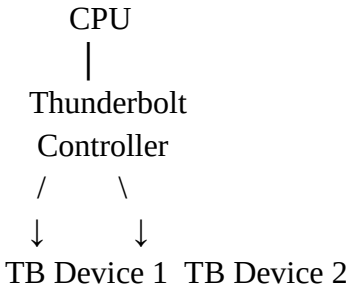
- Interrupt starvation.
- Bus monopolization.
- Unfair device access.

It must support:

- Hardware interrupts.
- Software interrupts.
- Fair bus arbitration.

2. Proposed Architecture





3. Hardware Interrupts

Hardware interrupts are generated when devices require immediate processor attention.
Examples:

- USB transfer completion.
- Thunderbolt event.
- Device error.
- DMA completion.

High-priority hardware interrupts should receive appropriate priority levels.

4. Software Interrupts

Software interrupts can be used for deferred processing.
For example:
Hardware IRQ
↓
Short ISR
↓
Software Interrupt / Deferred Work
↓
Driver Processing
The hardware ISR should not perform lengthy processing.

5. Avoiding Interrupt Starvation

Interrupt starvation can occur if one high-frequency device continuously occupies CPU time.
Use:
Priority levels
Critical real-time event
↓
Thunderbolt completion
↓
USB high-priority device
↓
USB normal device
↓
Background software interrupt
Interrupt moderation
Multiple device events can be grouped before notifying the CPU.
Bounded ISR execution
Each ISR should execute for a limited amount of time.
Deferred processing

Long processing should occur outside the hardware ISR.

6. Fair Bus Arbitration

A suitable arbitration method is weighted round-robin or round-robin arbitration.
Example:
USB1
↓
USB2
↓
USB3
↓
USB4
↓
USB5
↓
USB6
↓
TB1
↓
TB2
↓
Repeat
Weighted arbitration can give high-speed devices more bandwidth without allowing them to monopolize the bus.
For example:

Device	Priority/Weight
USB 1	1
USB 2	1
USB 3	1
USB 4	1
USB 5	1
USB 6	1
Thunderbolt 1	3
Thunderbolt 2	3

The exact weights should be determined from workload requirements.

7. Justification

The combination of:
priority-based interrupts + deferred software interrupts + interrupt moderation + fair bus arbitration
prevents a high-speed device from starving lower-speed peripherals while still allowing Thunderbolt devices to receive appropriate bandwidth.

Q4. Data Acquisition System at 500 MB/s

1. Problem Understanding

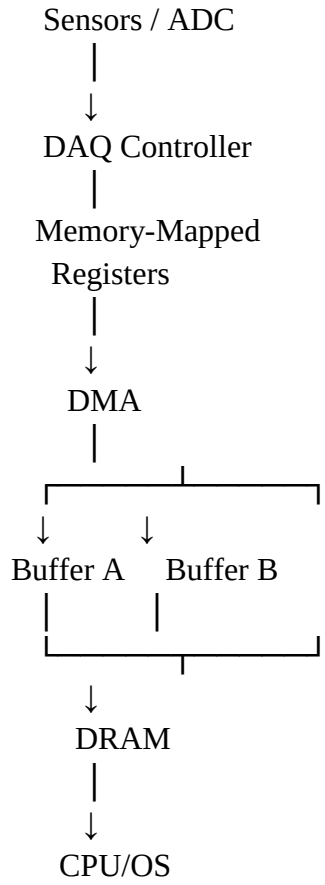
The data acquisition system continuously generates 500 MB/s of data.
The system requires:

- Continuous data capture.
- 500 MB/s sustained throughput.
- Minimal CPU intervention.
- Memory-mapped device registers.
- Comparison between DMA and interrupt-driven I/O.

2. Constraint Analysis

At:
[
500\ MB/s
]
the amount of data generated per second is extremely large for CPU-driven copying.
If the CPU handled every transfer using interrupts, interrupt frequency could become very high.
Therefore, the system should use DMA as the primary data-transfer mechanism.

3. Proposed Architecture



4. Memory-Mapped I/O

The device registers are mapped into the processor's address space.
For example:
DAQ_BASE + 0x00 → Control Register
DAQ_BASE + 0x04 → Status Register
DAQ_BASE + 0x08 → DMA Address
DAQ_BASE + 0x0C → DMA Length

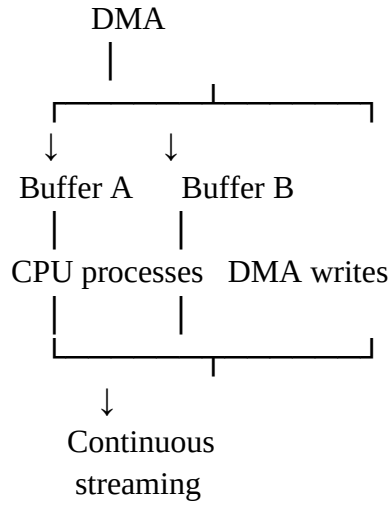
The CPU can configure the device using normal memory access instructions.

5. DMA vs Interrupt-Driven I/O

Feature	DMA	Interrupt-Driven I/O
CPU involvement	Low	High
Large data transfer	Excellent	Poor
Throughput	High	Lower for very frequent transfers
Interrupt frequency	Low/moderate	Potentially very high
CPU utilization	Low	High
Best application	Continuous high-speed streams	Small/occasional transfers
500 MB/s suitability	Excellent	Poor

6. Double Buffering

Double buffering prevents data loss while the CPU processes previously captured data.



When DMA fills Buffer A, it switches to Buffer B.
The CPU processes Buffer A while DMA fills Buffer B.

7. Throughput Analysis

Required rate:

[
R=500\ MB/s
]

For a 100 MB buffer:

[
Transfer\ Time=\frac{100}{500}=0.2\ seconds
]

Therefore, a 100 MB buffer would be filled every approximately 200 ms at a sustained 500 MB/s rate.

Buffer size should be selected according to acceptable latency, available memory, and processing time.

8. Justification

DMA is clearly preferable because the CPU only configures transfers and processes completed buffers instead of moving every sample.
The combination of:
Memory-mapped registers + DMA + double buffering + completion interrupts
provides continuous data acquisition with minimal CPU intervention.

Q5. Nested Interrupt Handling Mechanism

1. Problem Understanding

The system must support:

- Nested interrupts.
- Maximum interrupt latency below 10 µs.
- Vectored interrupts for high-priority devices.
- Software interrupts for background tasks.

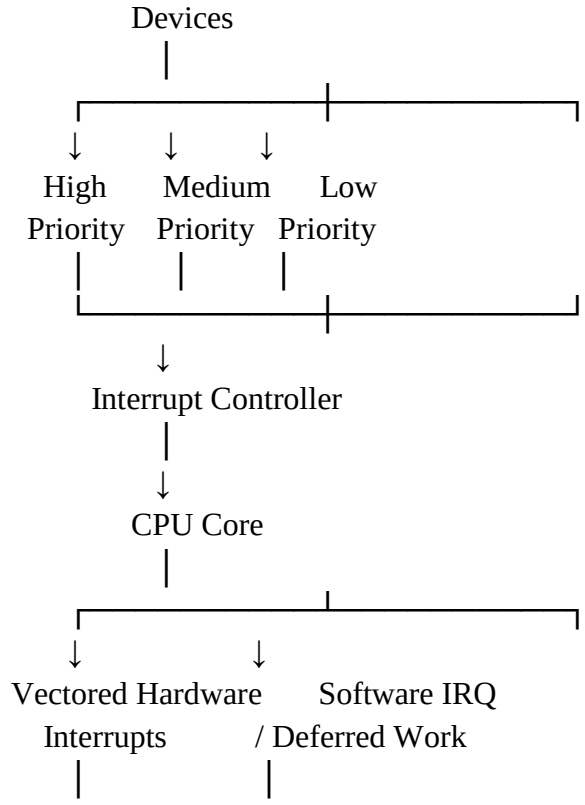
This is a real-time interrupt architecture where critical events must preempt lower-priority activities.

2. Constraint Analysis

Constraint	Design
Nested interrupts	Priority-based interrupt controller
Latency <10 µs	Short ISRs
High-priority devices	Vectored interrupts
Background tasks	Software interrupts/deferred work
Real-time behavior	Priority and bounded execution

The most important constraint is the <10 µs maximum interrupt latency.

3. Proposed Architecture



↓ ↓
Short ISRs Background Tasks

4. Nested Interrupt Operation

Suppose a low-priority interrupt is executing:

Low Priority ISR
↓
High Priority Interrupt
↓
Save Current Context
↓
High Priority ISR
↓
Restore Low Priority Context
↓
Continue Low Priority ISR

Thus, a critical event can interrupt a less important event.

5. Priority Scheme

Example:

Priority	Event	Handling
1	Emergency/critical device	Vectored hardware ISR
2	High-speed sensor	Vectored hardware ISR
3	Communication event	Hardware ISR
4	DMA completion	Hardware ISR
5	Background processing	Software interrupt

Lower numerical value can represent higher priority.

6. Achieving <10 μs Latency

Interrupt latency can be approximated as:

[
T_{latency} =
T_{detection}
+
T_{entry}
+
T_{context}
+
T_{dispatch}
]

The design must satisfy:

[
T_{latency}<10\mu s
]

Strategies include:

1. Vectored interrupts
Directly identify the correct ISR.
2. Short ISRs
Avoid complex computation.
3. Nested priority handling
Allow critical interrupts to preempt lower-priority handlers.
4. Disable interrupts only briefly
Long critical sections can violate the latency requirement.
5. Deferred processing
Move non-critical processing to software interrupts/tasks.
6. Fast interrupt controller
Use hardware interrupt prioritization and rapid vector dispatch.

7. Example Timing

Suppose a high-priority device generates an interrupt.

0 µs Device generates IRQ
 │
 │ 2 µs Interrupt detection/vector dispatch
↓
2 µs High-priority ISR starts
 │
 │ 3 µs Critical processing
↓
5 µs ISR completes

The approximate response latency is:

[
5\mu s < 10\mu s
]

The actual value must be validated on the target hardware using worst-case measurements rather than assumed from the architecture.

8. Software Interrupts

Background operations should not execute inside high-priority hardware ISRs.
Instead:

Hardware IRQ
 ↓
Short ISR
 ↓
Set flag / schedule software interrupt
 ↓
Software ISR / Deferred Task
 ↓
Background Processing

Examples:

- Logging.
- Statistics.
- Non-critical protocol processing.

- Buffer management.
- Diagnostics.

9. Justification

The proposed architecture satisfies all constraints:
Nested interrupts → priority-based interrupt controller
<10 μs latency → short, bounded, vectored ISRs
High-priority devices → hardware vectored interrupts
Background tasks → software interrupts/deferred processing
This architecture ensures that critical events receive immediate CPU attention without allowing background processing to compromise real-time responsiveness.